

02_enriquecimiento_ranking

May 9, 2026

1 Feature Engineering — Capa 2: Enriquecimiento con Ranking SCVS

Objetivo: Enriquecer las 8 features SRI con 6 variables financieras del ranking SCVS, para el subconjunto de empresas que tienen expediente registrado en la Superintendencia de Compañías.

1.1 Flujo de este notebook

1. Cargar `features_capa1.csv` (175 empresas con datos SRI)
2. Obtener el EXPEDIENTE SCVS para cada empresa usando `directorio_companias.xlsx`
3. Cargar `bi_ranking.csv` y filtrar por las empresas de FPA
4. Tomar el **último año fiscal disponible** por empresa
5. Construir las 6 features financieras
6. Exportar `features_capa2.csv`

Recordatorio: Este dataset (N 93) es el modelo de **validación de robustez**, no el modelo principal. El modelo principal usa `features_capa1.csv` (N=175).

```
[1]: # Imports
import pandas as pd
import numpy as np
import warnings
from pathlib import Path

warnings.filterwarnings('ignore')

# Rutas base
ROOT = Path('e:/TESIS MAESTRIA/Desarrollo_clustering_maestria')
PATH_C1 = ROOT / '03_feature_engineering/outputs/features_capa1.csv'
PATH_DIR = ROOT / '02_data_cleaning/data_super_compañias/'
    ↳ directorio_companias.xlsx'
PATH_RANKING = ROOT / '02_data_cleaning/data_super_compañias/Ranking/bi_ranking.'
    ↳ csv'
PATH_OUTPUT = ROOT / '03_feature_engineering/outputs'

print('Rutas configuradas correctamente.')
```

Rutas configuradas correctamente.

1.2 1. Cargar features_capa1.csv

```
[2]: # Cargar Capa 1 - base con las 8 features SRI
capa1 = pd.read_csv(PATH_C1, dtype={'RUC': str})
capa1['RUC'] = capa1['RUC'].astype(str).str.strip().str.zfill(13)

print(f'Empresas en Capa 1: {len(capa1)}')
print(f'Columnas: {list(capa1.columns)}')
```

Empresas en Capa 1: 167

Columnas: ['name_norm', 'RUC', 'source_label', 'source_winner', 'es_cliente_fpa', 'tipo_sociedad', 'obligado_contabilidad', 'es_agente_retencion', 'es_contribuyente_especial', 'estado_activo', 'antiguedad_anos', 'sector_ciiu_macro', 'region']

1.3 2. Obtener EXPEDIENTE SCVS desde el directorio de compañías

El directorio de la SCVS contiene el par RUC EXPEDIENTE. El expediente es el identificador interno de la SCVS — necesario para cruzar con bi_ranking.csv.

Nota técnica: el archivo tiene encabezados a partir de la fila 5 (skiprows=4).

```
[3]: # Cargar directorio SCVS (skiprows=4 porque las primeras 4 filas son metadata)
directorio = pd.read_excel(PATH_DIR, skiprows=4, header=0, dtype=str)
directorio.columns = [str(c).strip() for c in directorio.columns]
directorio['RUC'] = directorio['RUC'].astype(str).str.strip()
directorio['EXPEDIENTE'] = directorio['EXPEDIENTE'].astype(str).str.strip()

print(f'Total empresas en directorio SCVS: {len(directorio):,}')
print(f'Columnas disponibles: {list(directorio.columns[:8])}')
print()

# Filtrar solo las empresas de FPA y obtener RUC EXPEDIENTE
rucs_fpa = set(capa1['RUC'].tolist())
dir_fpa = (
    directorio[directorio['RUC'].isin(rucs_fpa)]
    [['RUC', 'EXPEDIENTE']]
    .drop_duplicates('RUC')
    .copy()
)

print(f'Empresas FPA con expediente SCVS: {len(dir_fpa)} / {len(capa1)}')
print(f'Empresas FPA SIN expediente: {len(capa1) - len(dir_fpa)}')
```

Total empresas en directorio SCVS: 219,339

Columnas disponibles: ['No. FILA', 'EXPEDIENTE', 'RUC', 'NOMBRE', 'SITUACIÓN LEGAL', 'FECHA_CONSTITUCION', 'TIPO', 'PAÍS']

Empresas FPA con expediente SCVS: 91 / 167

Empresas FPA SIN expediente: 76

1.4 3. Cargar bi_ranking.csv y filtrar por expedientes FPA

bi_ranking.csv contiene el historial financiero anual de todas las empresas registradas en la SCVS. Filtramos solo los expedientes de nuestras empresas para no procesar los 1.6M de filas completas.

```
[4]: # Cargar ranking en chunks para evitar OOM - filtramos por expediente en cada chunk
      print('Cargando bi_ranking.csv en chunks (puede tardar ~30 segundos)...')

      expedientes_fpa = set(dir_fpa['EXPEDIENTE'].tolist())

      CHUNK_SIZE = 100_000
      chunks_fpa = []

      for chunk in pd.read_csv(PATH_RANKING, low_memory=False, chunksize=CHUNK_SIZE):
          chunk['expediente'] = chunk['expediente'].astype(str).str.replace('.0', '').str.strip()
          filtrado = chunk[chunk['expediente'].isin(expedientes_fpa)]
          if len(filtrado) > 0:
              chunks_fpa.append(filtrado)

      ranking_fpa = pd.concat(chunks_fpa, ignore_index=True) if chunks_fpa else pd.DataFrame()
      ranking_fpa['anio'] = pd.to_numeric(ranking_fpa['anio'], errors='coerce')

      print(f'Filas de nuestras empresas en ranking: {len(ranking_fpa):,}')
      print(f'Empresas únicas con historial en ranking: {ranking_fpa["expediente"].nunique():}')
      print(f'Años disponibles: {sorted(ranking_fpa["anio"].dropna().unique().astype(int).tolist())}')

```

Cargando bi_ranking.csv en chunks (puede tardar ~30 segundos)...

Filas de nuestras empresas en ranking: 1,023

Empresas únicas con historial en ranking: 87

Años disponibles: [2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]

1.5 4. Tomar el último año fiscal disponible por empresa

El ranking tiene hasta 17 años de historia por empresa. Para el clustering usamos el estado **actual** del negocio, no el histórico. Se toma la fila más reciente por expediente.

```
[5]: # Ordenar por año descendente y quedarse con la fila más reciente por empresa
      rk_last = (
          ranking_fpa
          .sort_values('anio', ascending=False)

```

```

        .drop_duplicates(subset='expediente', keep='first')
        .copy()
    )

    print(f'Empresas con último balance disponible: {len(rk_last)}')
    print()
    print('Distribución del último año fiscal por empresa:')
    print(rk_last['anio'].value_counts().sort_index(ascending=False).head(6).
        ↪to_dict())
    print()

    # Verificar qué columnas financieras existen en el dataset
    print('Columnas disponibles en ranking:', list(rk_last.columns[:20]))
    print()

    # Convertir columnas financieras a numérico (si existen)
    COLS_FINANCIERAS = [
        'n_empleados', 'ingresos_ventas', 'activos', 'patrimonio',
        'cod_segmento', 'liquidez_corriente', 'margen_operacional'
    ]
    disponibles = [c for c in COLS_FINANCIERAS if c in rk_last.columns]
    faltantes = [c for c in COLS_FINANCIERAS if c not in rk_last.columns]
    print(f'Columnas encontradas: {disponibles}')
    print(f'Columnas NO encontradas: {faltantes}')

    for col in disponibles:
        rk_last[col] = pd.to_numeric(rk_last[col], errors='coerce')

    print()
    print('Estadísticas de variables financieras (antes de transformar):')
    for col in ['n_empleados', 'ingresos_ventas', 'activos']:
        if col in rk_last.columns:
            s = rk_last[col]
            print(f'    {col}: min={s.min():.0f}, mediana={s.median():.0f}, max={s.
                ↪max():.0f}, nulos={s.isna().sum()}')

```

Empresas con último balance disponible: 87

Distribución del último año fiscal por empresa:

{2025: 66, 2024: 14, 2022: 1, 2021: 1, 2011: 4, 2009: 1}

Columnas disponibles en ranking: ['anio', 'expediente', 'posicion_general', 'cia_imvalores', 'id_estado_financiero', 'ingresos_ventas', 'activos', 'patrimonio', 'utilidad_an_imp', 'impuesto_renta', 'n_empleados', 'ingresos_totales', 'utilidad_ejercicio', 'utilidad_neta', 'cod_segmento', 'ciiu_n1', 'ciiu_n6', 'liquidez_corriente', 'prueba_acida', 'end_activo']

Columnas encontradas: ['n_empleados', 'ingresos_ventas', 'activos',

'patrimonio', 'cod_segmento', 'liquidez_corriente', 'margen_operacional']
 Columnas NO encontradas: []

Estadísticas de variables financieras (antes de transformar):

n_empleados: min=1, mediana=33, max=2877, nulos=0
 ingresos_ventas: min=0, mediana=11376870, max=662335105, nulos=0
 activos: min=0, mediana=13471621, max=502093690, nulos=0

1.6 5. Construcción de las 6 features financieras

#	Feature	Columna origen	Transformación	Justificación
9	log_empleados	n_empleados	log (x+1)	Distribución muy sesgada — log comprime la escala
10	log_ingresos	ingresos_ventas	log (x+1)	Rango: 0 a 662M — imposible sin log
11	log_activos	activos	log (x+1)	Mismo problema de escala
12	segmento	cod_segmento	ordinal 1–4 (micro→grande)	Clasificación oficial SCVS
13	liquidez_corriente	liquidez_corriente	clip p99	Ratio de solvencia de corto plazo
14	margen_operacional	margen_operacional	clip p99	Eficiencia operativa del negocio

```
[6]: # Features 9, 10, 11: logarítmicas
# log (x+1): el +1 evita log(0) en empresas con valor = 0
rk_last['log_empleados'] = np.log10(rk_last['n_empleados'].clip(lower=0) + 1)
rk_last['log_ingresos'] = np.log10(rk_last['ingresos_ventas'].clip(lower=0) + 1)
rk_last['log_activos'] = np.log10(rk_last['activos'].clip(lower=0) + 1)

# Feature 12: segmento SCVS
# 1=Microempresa, 2=Pequeña, 3=Mediana, 4=Grande
rk_last['segmento'] = rk_last['cod_segmento'].astype(float)

# Features 13, 14: indicadores financieros con clip en p99
# El clip elimina outliers extremos que distorsionarían el clustering.
# p99 = se conserva el 99% de la distribución real, solo se recortan los casos extremos.
p99_liq = rk_last['liquidez_corriente'].quantile(0.99)
p99_mar = rk_last['margen_operacional'].quantile(0.99)
```

```

p01_mar = rk_last['margen_operacional'].quantile(0.01) # también clip inferior
↳ para márgenes negativos extremos

rk_last['liquidez_corriente_clip'] = rk_last['liquidez_corriente'].
↳ clip(lower=0, upper=p99_liq)
rk_last['margen_operacional_clip'] = rk_last['margen_operacional'].
↳ clip(lower=p01_mar, upper=p99_mar)

print(f'Clip liquidez_corriente en p99: {p99_liq:.2f}')
print(f'Clip margen_operacional en [p01={p01_mar:.2f}, p99={p99_mar:.2f}]')
print()

# Resumen de las 6 features financieras
FEATURES_FIN = [
    ('log_empleados', rk_last['log_empleados']),
    ('log_ingresos', rk_last['log_ingresos']),
    ('log_activos', rk_last['log_activos']),
    ('segmento', rk_last['segmento']),
    ('liquidez_corriente', rk_last['liquidez_corriente_clip']),
    ('margen_operacional', rk_last['margen_operacional_clip']),
]

print('=== Estadísticas de las 6 features financieras ===')
for nombre, serie in FEATURES_FIN:
    print(f' {nombre}: min={serie.min():.2f}, mediana={serie.median():.2f}, '
          f'max={serie.max():.2f}, nulos={serie.isna().sum()}')

print()
print('Distribución segmento (1=Micro, 2=Pequeña, 3=Mediana, 4=Grande):')
print(rk_last['segmento'].value_counts().sort_index().to_dict())

```

Clip liquidez_corriente en p99: 4093.84

Clip margen_operacional en [p01=-1331.53, p99=6316.73]

=== Estadísticas de las 6 features financieras ===

log_empleados: min=0.30, mediana=1.53, max=3.46, nulos=0

log_ingresos: min=0.00, mediana=7.06, max=8.82, nulos=0

log_activos: min=0.00, mediana=7.13, max=8.70, nulos=0

segmento: min=1.00, mediana=4.00, max=4.00, nulos=1

liquidez_corriente: min=0.00, mediana=1.46, max=4093.84, nulos=5

margen_operacional: min=-1331.53, mediana=0.00, max=6316.73, nulos=5

Distribución segmento (1=Micro, 2=Pequeña, 3=Mediana, 4=Grande):

{1.0: 27, 2.0: 6, 3.0: 7, 4.0: 46}

1.7 6. JOIN Capa 1 ← features financieras y exportar features_capa2.csv

```
[7]: # Preparar tabla de features financieras con RUC como llave
rk_export = rk_last[['expediente', 'anio', 'log_empleados', 'log_ingresos',
                    'log_activos', 'segmento',
                    'liquidez_corriente_clip', 'margen_operacional_clip']].
    ↪copy()
rk_export = rk_export.rename(columns={
    'liquidez_corriente_clip': 'liquidez_corriente',
    'margen_operacional_clip': 'margen_operacional'
})

# Agregar RUC al ranking a través del directorio
rk_export = rk_export.merge(
    dir_fpa.rename(columns={'EXPEDIENTE': 'expediente'}),
    on='expediente',
    how='left'
)

# JOIN Capa 1 + features financieras (LEFT: conservamos las 175 de Capa 1)
capa2 = capa1.merge(rk_export.drop(columns='expediente'), on='RUC', how='left')

# Diagnóstico
con_financieras = capa2['log_ingresos'].notna().sum()
sin_financieras = capa2['log_ingresos'].isna().sum()

print(f'Total empresas en Capa 2 (completo):    {len(capa2)}')
print(f'  Con features financieras (ranking):    {con_financieras}')
print(f'  Sin features financieras (solo SRI):    {sin_financieras}')
print()

# Para el modelo de Capa 2 solo usamos las que tienen datos financieros
↪completos
capa2_modelo = capa2[capa2['log_ingresos'].notna()].copy().
    ↪reset_index(drop=True)
print(f'Dataset Capa 2 (solo con datos financieros): {len(capa2_modelo)}')
↪empresas')
print(f'Clientes FPA en Capa 2: {capa2_modelo["es_cliente_fpa"].sum()} /')
↪{len(capa2_modelo)}')
print()

# Diagnóstico de nulos en features financieras
COLS_FIN = ['log_empleados', 'log_ingresos', 'log_activos',
            'segmento', 'liquidez_corriente', 'margen_operacional']
print('=== Nulos en features financieras (sobre las', len(capa2_modelo),
    ↪'empresas) ===')
for col in COLS_FIN:
```

```

n_null = capa2_modelo[col].isna().sum()
estado = 'OK' if n_null == 0 else f'{n_null} NULOS'
print(f' {col}: {estado}')

# Exportar (versión completa con NaN para las sin ranking, útil para análisis)
out_path = PATH_OUTPUT / 'features_capa2.csv'
capa2_modelo.to_csv(out_path, index=False)
print(f'\nExportado: {out_path}')
print(f'Shape: {capa2_modelo.shape}')
print(f'Columnas: {list(capa2_modelo.columns)}')

```

```

Total empresas en Capa 2 (completo):    167
  Con features financieras (ranking):    87
  Sin features financieras (solo SRI):    80

```

```

Dataset Capa 2 (solo con datos financieros): 87 empresas
Clientes FPA en Capa 2: 37 / 87

```

```

=== Nulos en features financieras (sobre las 87 empresas) ===
log_empleados: OK
log_ingresos: OK
log_activos: OK
segmento: 1 NULOS
liquidez_corriente: 5 NULOS
margen_operacional: 5 NULOS

```

```

Exportado: e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\03_feature_engineeri
ng\outputs\features_capa2.csv
Shape: (87, 20)
Columns: ['name_norm', 'RUC', 'source_label', 'source_winner',
'es_cliente_fpa', 'tipo_sociedad', 'obligado_contabilidad',
'es_agente_retencion', 'es_contribuyente_especial', 'estado_activo',
'antiguedad_anos', 'sector_ciiu_macro', 'region', 'anio', 'log_empleados',
'log_ingresos', 'log_activos', 'segmento', 'liquidez_corriente',
'margen_operacional']

```